Hochat Online

3-Hour Backend Study Plan for the Defense · Group 2

The professor focuses on the **backend**, so this plan skips React, Inertia, Wayfinder and styling.
Do the blocks in order and stop each one when its time is up. Tick the ☐ boxes as you go.

0:00 – 0:15 · How a request travels through the backend (15 min)

```
public/index.php → bootstrap/app.php → routes (web.php / api.php)
   → middleware (auth, role, CSRF) → Form Request (validation)
   → controller → model (Eloquent) → MySQL → response (page data or JSON)
```

- `bootstrap/app.php`: loads both route files, registers the `role` alias, and makes `/api` errors return JSON.
- Run `php artisan route:list --except-vendor` and read every route with its middleware.

☐ **Check:** explain the chain above for "staff updates an order status."

0:15 – 0:45 · Database and models (30 min)

File	Know this
`database/migrations/*`	Data types, `foreignId()->constrained()`, `unique()`, `cascadeOnDelete()`, `enum` for role / status / order_type, `delivery_fee` added in its own migration
`app/Models/User.php`	`$fillable` (mass-assignment protection), `'password' => 'hashed'`, the safe `role()` accessor, the `orders()` relationship
`app/Models/Order.php`	`belongsTo(User)`, `hasMany(OrderItem)`, the `casts()`
`database/seeders/`	The 3 sample accounts and the 85-item real menu

Practise in tinker (`php artisan tinker`):

```
App\Models\User::where('email', 'user@hochat.test')->first()->orders;
App\Models\Order::with('orderItems.menuItem')->latest()->first();
```

☐ **Check:** draw the 5 tables and their relationships from memory.

0:45 – 1:25 · Business logic and validation ⭐ (40 min)

1. `Order::placeFor()` (`app/Models/Order.php`), line by line:
   - empty cart → error
   - Delivery: needs an address containing "Gerona", plus the ₱50 fee
   - sold-out item → error
   - errors are thrown as `ValidationException` → **422**
   - `DB::transaction` saves the order and its items together, or not at all
2. **Form Requests** (`app/Http/Requests/`):
   - `PlaceOrderRequest`: order type, scheduled time must be after now
   - `MenuItemRequest`: price rules, photo upload (`image`, `max:2048`)
   - `UpdateOrderStatusRequest`: only the allowed statuses
3. `Admin/OrderController::update()`: Completed or Cancelled orders can't be changed.
4. `Admin/DashboardController`: the report queries: `groupBy('status')` counts, and sales from **Completed** orders only.

☐ **Check:** for each rule, say which file enforces it and what the user gets when it's broken (**422** + message).

1:25 – 2:00 · Security (35 min)

Topic	Where
Authentication	`LoginController`: `Auth::attempt`, `session()->regenerate()`; logout `invalidate()` + `regenerateToken()`
Password hashing	`'password' => 'hashed'` (bcrypt); `Hash::check` in the API login
CSRF	Automatic on `web.php` (`PreventRequestForgery`) → **419**; the API uses tokens instead
Roles	`EnsureUserHasRole` (**try-catch**, **403**) + the `role:` groups in the routes
Ownership	`OrderPolicy`: own orders only; cancel within 60 s while Pending
Brute force	`throttle:5,1` on login and forgot-password
SQL injection	Eloquent uses prepared statements
Mass assignment	`$fillable` on every model

Practise the panel's test:

- ☐ Delete `case Staff = 'Staff';` in `app/Enums/Role.php`
- ☐ Log in as `staff@hochat.test` → a message appears, no crash
- ☐ Log in as admin → the Report still opens
- ☐ Open `storage/logs/laravel.log` → the warning lines are there
- ☐ **Put the Staff line back** (`git checkout app/Enums/Role.php`)

2:00 – 2:35 · REST API (35 min)

1. `routes/api.php`: the prefixes in `/api/v1/admin/orders`, `auth:sanctum`, and the `role:` groups. The prefix only builds the URL; the middleware protects it.
2. `Api/V1/AuthController`: `createToken()` on login; logout deletes the token.
3. `app/Http/Resources/`: how the JSON responses are shaped (`OrderResource`, `MenuItemResource`).
4. **Status codes:**

200	201	204	401	403	404	422
OK	Created	Deleted	No / bad token	Wrong role	Not found	Invalid data

5. **Postman** (`Hochat-Online.postman_collection.json`): log in → cart → checkout → staff updates the status → show a **403** and a **422**. Press **Ctrl + Alt + C** to show exactly what was sent. Run **Logout** last.

☐ **Check:** say what an admin token gets on `GET /api/v1/orders` (403, customers only) and why.

2:35 – 3:00 · Debugging and tests (25 min)

The panel gave another group a **live debugging** task, so practise these:

☐	Scenario	What to do
☐	Remove a role	Already handled (try-catch). Show the login message and the log.
☐	A 500 error appears	Open `storage/logs/laravel.log` → read the **file and line** at the top
☐	"Why is this wrong?"	`dd($variable);` in the controller → look → **remove it**
☐	Check the data	`php artisan tinker`
☐	"Does this URL exist? Who can use it?"	`php artisan route:list --path=orders`
☐	MySQL is off	"Connection refused" → start MySQL in Laragon
☐	Prove the rules work	`php artisan test --compact` → 85 passed
☐	Change a rule	Set `DELIVERY_FEE` to `60` → run the tests → watch one fail → change it back

Questions a backend-focused professor will likely ask

☐	Question	Answer with
☐	"Walk me through what happens when an order is placed."	`Order::placeFor()`
☐	"How do you stop a customer from opening admin pages?"	The `role` middleware → 403
☐	"How do you stop one customer from seeing another's order?"	`OrderPolicy`
☐	"How are passwords stored?"	bcrypt hash, never plain text
☐	"What protects you from SQL injection / CSRF?"	Eloquent prepared statements / `PreventRequestForgery`
☐	"What if two things must save together?"	`DB::transaction`
☐	"Where is your error handling?"	try-catch in the role middleware and `Role::fromStored`; `ValidationException`; JSON errors in `bootstrap/app.php`
☐	"Show your REST API."	`api.php` + Postman + status codes
☐	"Show me your database relationships."	Migrations + `hasMany` / `belongsTo`
☐	"Did you write the CSRF protection?"	"No, Laravel provides it. We wrote the role-based access control."

From the lectures: what the professor taught, and where it is in Hochat

The lectures teach these topics in **plain PHP** (PDO, `$_POST`, `password_hash()`, `session_start()`). Hochat does the same things with **Laravel**. In the defense: *"We learned it in plain PHP; here is the Laravel version in our system."*

Topic 4 · Authentication, Authorization and Backend Security

Lecture says	In Hochat
AuthN vs AuthZ: who you are vs what you may do	`auth` middleware (logged in?) vs `role:` middleware + `OrderPolicy` (allowed?)
Typical auth flow: submit → validate → find account → verify password → session/token → check on every request	`LoginController::store`: `validate()` → `Auth::attempt()` → `session()->regenerate()` → `auth` middleware on later pages
Hashing vs encryption vs salting: use a slow adaptive hash (bcrypt/Argon2), never MD5/SHA1 or encryption	`'password' => 'hashed'` in `User.php` uses **bcrypt**, which includes its own salt
`password_hash()` / `password_verify()`	Laravel's `Hash` wraps the same functions; `Hash::check()` in the API login
Secure session config: HttpOnly, Secure, SameSite=Lax, regenerate the ID after login	`config/session.php`: `http_only` and `same_site => 'lax'` by default; `regenerate()` after login
CSRF tokens on forms	Automatic on `web.php` (`PreventRequestForgery`) → 419
Secure logout: destroy the session	`Auth::logout()` + `invalidate()` + `regenerateToken()`
SQL injection: use prepared statements	Eloquent always binds values
Session vs token: session for web apps (risk: CSRF); token for APIs (risk: token theft)	Website = session; REST API = Sanctum token. Hochat shows **both**.
Principles: least privilege · defense in depth · never trust input · fail securely	Each role gets only its pages · route middleware *and* policy *and* validation · Form Requests on the server · 403 / login refused on unknown roles

Topic 5 · PHP REST API

Lecture says	In Hochat
Resources and endpoints: URLs name things, methods name actions	`/api/v1/menu`, `/cart`, `/orders/{order}` in `routes/api.php`
Safety and idempotency: GET is safe; GET, PUT and DELETE are idempotent (repeating gives the same result); POST is not	Sending **POST /cart** twice adds the item twice; **DELETE /cart/1** twice leaves the cart the same
Status codes must agree with the body: 200, 201, 204, 401, 403, 404, 422	201 on create, 204 on delete, 422 with `errors`, 401 / 403 / 404 from Laravel

`json_encode()` / reading JSON with `php://input`	API Resources build the JSON; `$request->validated()` reads JSON bodies
Validate before you write	Form Requests in `app/Http/Requests/Api/`
Standardized error responses, dev vs production	`bootstrap/app.php` forces JSON errors on `/api`; `APP_DEBUG=false` hides the trace in production
Bearer token check and roles	`auth:sanctum` + `role:Staff,Administrator`
Secure file upload: size limit, MIME check, store safely	`'photo' => ['nullable', 'image', 'max:2048']` + `store('menu', 'public')`
401 vs 403: 401 = who you are, 403 = what you may do	No token → 401 · admin token on `/api/v1/orders` → 403

"Verify the API, don't just look at the response" (lecture checklist): ☐ the status code is the expected one · ☐ the body has the right shape · ☐ the database changed (check phpMyAdmin) · ☐ a failed request changed nothing · ☐ repeat POST and DELETE and compare

The lecture's debugging challenge, and how Hochat avoids each mistake

Mistake in the lecture's broken endpoint	Hochat
User input put straight into the SQL string (SQL injection)	Eloquent binds every value
No validation of the input	Form Requests reject bad data with 422
Missing fields cause PHP warnings	`required` rules run before the controller
Always answers "success", with no status code	Real codes: 201, 422, 403, 404
No error handling	`ValidationException`, try-catch for roles, JSON error handler
No authentication / authorization	`auth:sanctum` + `role:` middleware

Weeks 2–3 · Intro to PHP and SQL

Lecture topic	In Hochat
Arrays, `foreach`, functions with return types	Session cart `[menu_item_id => quantity]`; every method has return types
Anonymous / arrow functions, callbacks	`fn (MenuItem $item) => …` in `Order::placeFor()`
Constants	`Order::DELIVERY_FEE`, `Order::DELIVERY_AREA`
`$_GET` / `$_POST`	`$request->validated()`; report dates and menu search use GET
Prepared statements, basic error handling	Eloquent; Laravel's error handler + `laravel.log`

Weeks 4–5 · Object-Oriented PHP

Lecture topic	In Hochat
Classes and objects, encapsulation	Models and controllers; `$fillable`, `protected` methods
Inheritance	`Order extends Model`; controllers extend the abstract `Controller`
Polymorphism	Every model inherits `create()`, `update()`, `delete()` from `Model`
Traits	`use HasApiTokens, HasFactory, Notifiable;` in `User.php`
Type declarations, enums	Typed parameters and returns; `enum Role: string`
Exception handling (try-catch)	`EnsureUserHasRole` and `Role::fromStored()` catch `ValueError`
Namespaces, Composer autoloading (PSR-4)	`namespace App\Http\Controllers\Admin;` · `"App\\": "app/"` in `composer.json`
PSR-12 coding standard	Laravel Pint formats the code
Interfaces	⚠ None written by the group. Be honest: "We used inheritance and an enum for roles."

Weeks 6–7 · MySQL Database

Lecture topic	In Hochat
Data types, keys and constraints	Migrations: `decimal(8,2)`, `enum`, `unique()`, `foreignId()->constrained()`
DDL · DML · DQL · TCL	DDL = migrations · DML = `create()` / `update()` / `delete()` · DQL = `where()`, `latest()`, `limit()` · TCL = `DB::transaction`
Last inserted ID	`$order->id` right after `create()`, used for `order_items`
SELECT / WHERE / ORDER BY / LIMIT	`Admin/DashboardController`: `where`, `latest()`, `limit(50)`, `groupBy('status')`
SQL injection	Prevented by Eloquent's bound parameters

Terms to be able to define in one sentence: authentication · authorization · hashing vs encryption · salt · session vs token · CSRF · SQL injection · prepared statement · idempotent · 401 vs 403 · 422 · least privilege · defense in depth · fail securely · transaction · foreign key · inheritance · trait · namespace · PSR-4.

Skip completely: everything in `resources/js/`, Tailwind and styling, Inertia, Wayfinder, and `vendor/` / `node_modules/`.

Only 1 hour? Do `Order::placeFor()` (0:45 – 1:25) and the remove-Staff practice. Those are the most likely questions.